

AI-Assisted Development Exercise — Tic-Tac-Toe, No Draws

Please complete the following exercise before the interview. If you can, please send us the link to the repo and transcript of your AI dev conversation before your interview, so that we can review it prior to your interview.

Purpose and time box

This exercise is not a test of whether you can write tic-tac-toe. We assume you can.

We are assessing **how you work with AI coding tools** — how you turn an underspecified requirement into something you can defend. Use whatever tooling you like: Claude Code, Codex, Gemini CLI, Cursor, Copilot, or a combination.

Time box: Maximum of 3 hours of active work. If you run over, stop and note the overrun and what caused it. We would rather see an honest partial result than an unbounded one.

You may use any model, any tool, any reference material. There is no requirement to work unaided — the opposite is the point.

The problem

Design and build a two-player tic-tac-toe variant **in which a game can never end in a draw**.

Rules of the variant are yours to invent. It must remain recognizably tic-tac-toe, but not limited to the followings: a 3×3 grid, two players, X and O, alternating turns.

Every game must satisfy both of:

- **No draw.** There is no terminal state in which neither player has won.
- **Always terminates.** There is no line of play that continues indefinitely.

The brief is deliberately underspecified in places. Where it is, make a decision, and record the decision and your reasoning. Do not email us to ask — how you handle the gap is part of what we are looking at. The point of this exercise is to determine if you can understand the intent with incomplete information.

Technical constraints

- HTML, CSS and JavaScript. Runs in a browser.
- **No build step.** `python -m http.server` in the repo root, open the page, play. If you use a framework, it must still serve as static files.
- No backend, no database, no network calls.
- Human vs human is sufficient. A computer opponent is optional and is not scored.
- Works in current Chrome. Nothing else required.

Deliverables

A single git repository containing:

1. **The game.** Playable as described above.
2. `docs/RULES.md` — the variant stated plainly enough that a stranger could play it from a printed copy, without reading your code.
3. `docs/DESIGN.md` — covering:
 - Your reading of the brief, including any ambiguity you found and how you resolved it
 - Rule sets you considered and rejected, and why
 - Your argument that a draw is impossible and that play always terminates. An informal argument is acceptable; a proof or an exhaustive search over the game tree is better
 - Anything you know is broken or unfinished
4. **Commit history.** Commit as you go, at natural boundaries. A single squashed commit at the end tells us nothing and counts against you.
5. `transcript/` — see the next section.

Send us a link to the repository, or a zip if you would rather not use a host.

Transcript and tooling evidence

We review the full record of how the work was produced. This is the part of the submission we spend the most time on.

What to supply

Put all of the following under `transcript/`:

The raw session files your tool writes to disk. Not a summary, not a recap you asked the model to write, not a curated log. The machine-generated record. Locating and exporting it is part of the exercise; most CLI tools keep sessions as JSON or JSONL under your home directory.

Every configuration file that shaped the model's behaviour, for example: `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, `.cursor/rules/`, `.github/copilot-instructions.md`, custom slash commands, agent or skill definitions, hooks, MCP server configuration. Redact any credentials before committing.

A short transcript/README.md listing which tools and models you used, which session file corresponds to which phase of work, and anything that did not get captured.

If you used multiple sessions or multiple tools

Include all of them, including the ones that went nowhere. Abandoned attempts are useful to us.

If your tool compacts or truncates context

That is your problem to solve, and how you solve it is itself informative. Copy the session file out mid-run, start fresh sessions at phase boundaries, or use a tool that preserves the record. Do not hand us a compacted summary and call it the transcript.

A note on honesty

We cross-check the transcript against commit timestamps. A messy, real transcript is worth far more to us than a tidy one that does not match the repository. If something went wrong, leave it in.

At the interview

Allow about 45 minutes:

- **Demo.** You run the game on your own machine and play a round with us.
- **Live change.** We will ask for one modification, unannounced, and you make it in front of us using your tools. This is not a trick — it is the same work you did at home, at smaller scale.
- **Walkthrough.** We go through your `DESIGN.md` and parts of your transcript together, and you talk us through the decisions.

Have your development environment ready to go before the call.

What is not scored

So you do not waste the time box:

- Visual design. A plain grid is fine.
- Animation, sound, responsive layout, accessibility polish.
- A computer opponent.
- Test coverage beyond whatever supports your no-draw and termination argument.
- Framework or language choice, within the constraints above.
- Whether your rule variant is *original*. It only has to be correct and defended.